



Table-Driven Testing

The most **idiomatic** way to write tests in Go

```
tests := []struct {
    name     string
    input    int
    want     int
}{
    // ...
}

for _, tt := range tests {
    t.Run(tt.name, func(t *testing.T) {
        got := Add(tt.input)
        if got != tt.want {
            t.Errorf("got %d, want %d",
                got, tt.want)
        }
    })
}
```



Chapter 1



Go Testing
Handbook

name	input	want	expected
adds positives	2	4	✓
adds zero	0	0	✓
adds negative	-3	-6	✓
mixed signs	-2	1	✓
...	...	...	...



Why Table-Driven Testing?

Instead of writing
20 similar test functions...

Write **ONE** test
for **MANY** scenarios.

```
func TestAdd_Positive(t *testing.T) {  
    got := Add(2, 4)  
    want := 6  
    if got != want {  
        t.Errorf("got %d, want %d",  
            got, want)  
    }  
}
```

```
func TestAdd_Zero(t *testing.T) {  
    got := Add(0, 0)  
    want := 0  
    if got != want {  
        t.Errorf("got %d, want %d",  
            got, want)  
    }  
}
```

⋮

```
func TestAdd_MixedSigns(t *testing.T) {  
    got := Add(-2, 1)  
    want := -1  
    if got != want {  
        t.Errorf("got %d, want %d",  
            got, want)  
    }  
}
```



```
func TestAdd(t *testing.T) {  
    tests := []struct {  
        name string  
        a, b int  
        want int  
    }{  
        {"adds positives", 2, 4, 6},  
        {"adds zero", 0, 0, 0},  
        {"adds negative", -3, -6, -9},  
        {"mixed signs", -2, 1, -1},  
        // ... more cases  
    }  
    for _, tt := range tests {  
        t.Run(tt.name, func(t *testing.T) {  
            got := Add(tt.a, tt.b)  
            if got != tt.want {  
                t.Errorf("got %d, want %d",  
                    got, tt.want)  
            }  
        })  
    }  
}
```



Hard to maintain.
Easy to miss cases.



Easy to add cases.
Easy to read & maintain.





3

Think of test cases as data.



Store every scenario in a table.



Run the same logic for each row.

test cases (data)

name	a	b	want	expected
adds positives	2	4	6	✓
adds zero	0	0	0	✓
adds negative	-3	-6	-9	✓
mixed signs	-2	1	-1	✓
...	...	...	...	...



```
for _, tc := range tests {  
    got := Add(tc.a, tc.b)  
    if got != tc.want {  
        t.Errorf("got %d, want %d",  
            got, tc.want)  
    }  
}
```



Data in, logic once, confidence out.



4

Why Go developers love this pattern

**Less** duplicated code

One test handles many cases.

**Easier** maintenance

Update logic in one place.

**Better** readability

Scenarios are clear and concise.

**Easy** to extend

Just add a new row.



```
func TestAdd(t *testing.T) {  
    tests := []struct {  
        a, b int  
        want int  
    }{  
        // ... test cases  
    }  
    for _, tc := range tests {  
        got := Add(tc.a, tc.b)  
        if got != tc.want {  
            t.Errorf("got %d, want %d",  
                got, tc.want)  
        }  
    }  
}
```



a	b	want	description	
2	4	6	adds positives	✓
0	0	0	adds zero	✓
-3	-6	-9	adds negative	✓
-2	1	-1	mixed signs	✓
...	...	...	...	✓

Write once. Test many. Ship with **confidence**.



5 How table-driven tests work



1

a	b	want
2	4	6
0	0	0
-3	-6	-9
...	...	...

Test Cases

Define a list of input and expected output.

2

```
for _, tc := range tests {  
    ...  
}
```

for ... range

Loop through each test case in the table.

3

```
got := Add(tc.a, tc.b)
```

Run Function

Call the function with the test case input.

4

```
if got != tc.want {  
    t.Errorf(",  
        \"got %d, want %d\",  
        got, tc.want,  
    )  
}
```

Compare Result

Compare the actual result with the expected value.



One test function. Many cases.
Simple, scalable, and powerful.

```
func TestAdd(t *testing.T) {  
    for _, tc := range tests {  
        ...  
    }  
}
```

6

Don't write one test function per scenario.

Write one test... with many scenarios.



Not ideal

Harder to maintain.
More code to write and run.

✗ `func TestAdd_Positive(t *testing.T) {
 ... // only positive case
}`

✗ `func TestAdd_Zero(t *testing.T) {
 ... // only zero case
}`

✗ `func TestAdd_Negative(t *testing.T) {
 ... // only negative case
}`



Slower to run



More boilerplate



Harder to extend

VS



Better

Easier to maintain.
Less code, more coverage.

```
func TestAdd(t *testing.T) {  
    tests := []struct {  
        a, b int  
        want int  
        desc string  
    }{  
        {2, 4, 6, "adds positives"},  
        {0, 0, 0, "adds zero"},  
        {-3, -6, -9, "adds negative"},  
        {-2, 1, -1, "mixed signs"},  
        // ... more cases  
    }  
    for _, tc := range tests {  
        got := Add(tc.a, tc.b)  
        if got != tc.want {  
            t.Errorf("got %d, want %d",  
                got, tc.want)  
        }  
    }  
}
```



Faster to run



More coverage in one place



Easy to extend with a new row



Think in tables.

Your tests will thank you.





You're all set!

Time to put table-driven testing into practice.



Read the full chapter on GitHub

Dive deeper into table-driven testing with examples, best practices, and more.



Link in bio

`>_ https://github.com/golang-evangelist/
.../01-table-driven-testing.md`



Save it. Star it. Share it! ★



Next: Subtests (t.Run)

Learn how to run each test case as a separate subtest for better reporting.

```
for _, tc := range tests {  
    t.Run(tc.name, func(t *testing.T) {  
        // test logic  
    })  
}
```



Keep learning. Keep testing.
You've got this! 💪